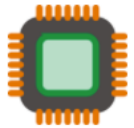


CASPER

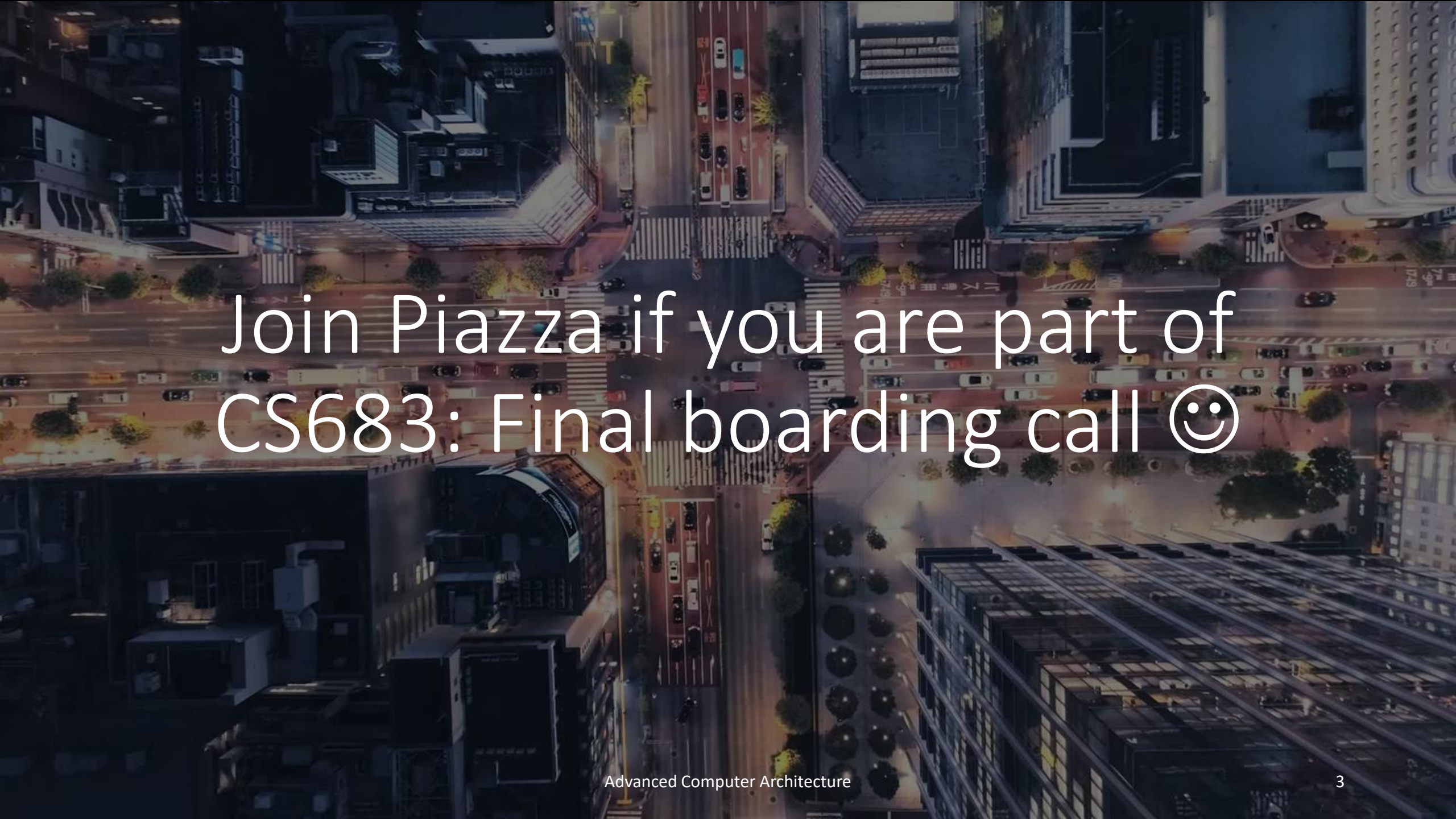
CS683: Advanced Computer Architecture-2024

L2-Catch the Cache

https://www.cse.iitb.ac.in/~biswa/courses.html

https://www.cse.iitb.ac.in/~biswa/

Phones (smart/non-smart) on silence plz, Thanks


Join Piazza if you are part of
CS683: Final boarding call ☺

Summary of Lecture-1

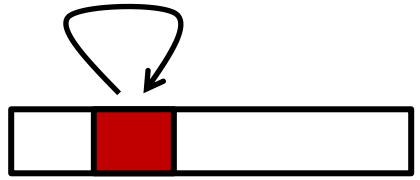
Caches

Hardware hash tables ☺

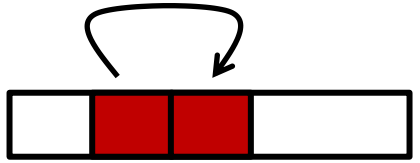
Catch the cache

Locality (why does it exist?)

- **Temporal locality:**
  - Recently referenced items are likely to be referenced again



- **Spatial locality:**
  - Items with nearby addresses tend to be referenced again



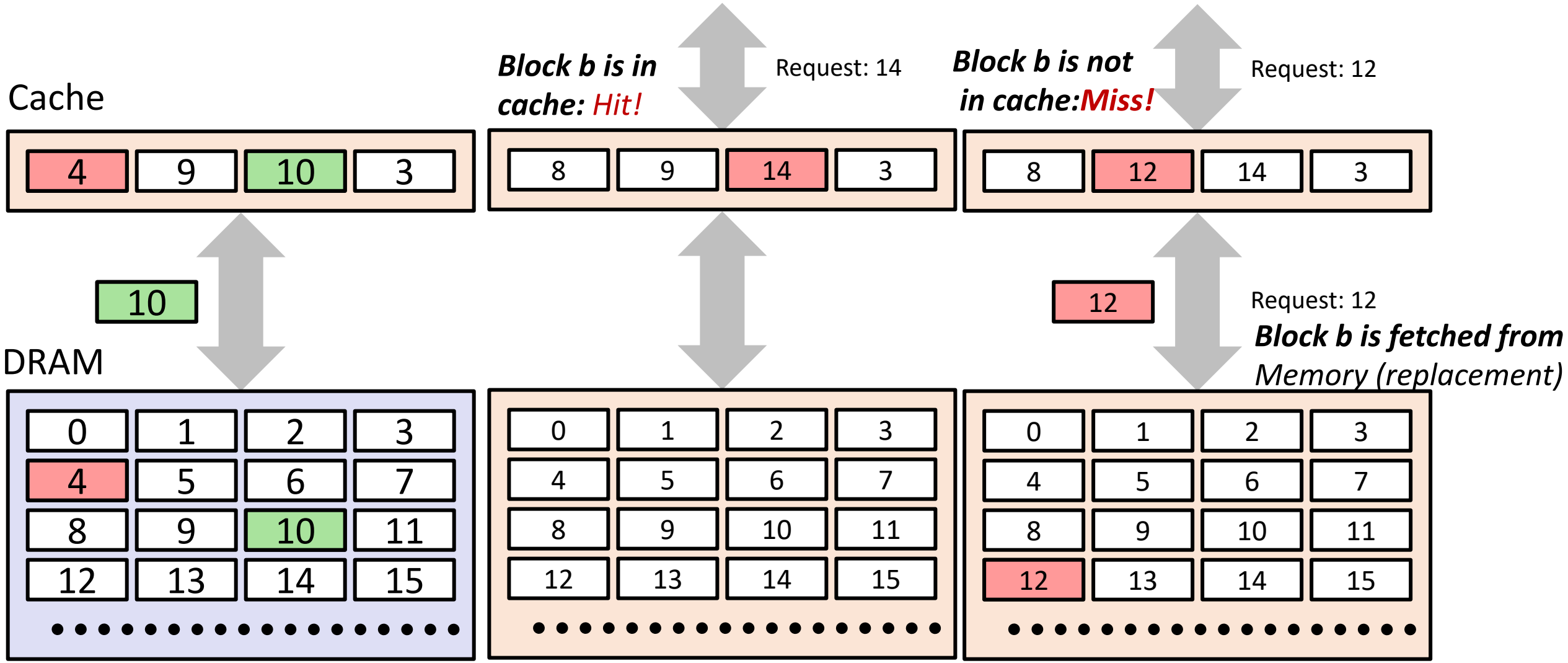
Locality: Example

```
sum = 0;
for (i = 0; i < n; i++)
  sum += a[i];
return sum;
```

Spatial/Temporal Locality?

- Data references
  - Reference array elements in succession (stride-1 reference pattern). — spatial
  - Reference variable **sum** each iteration. — temporal

Cache and DRAM

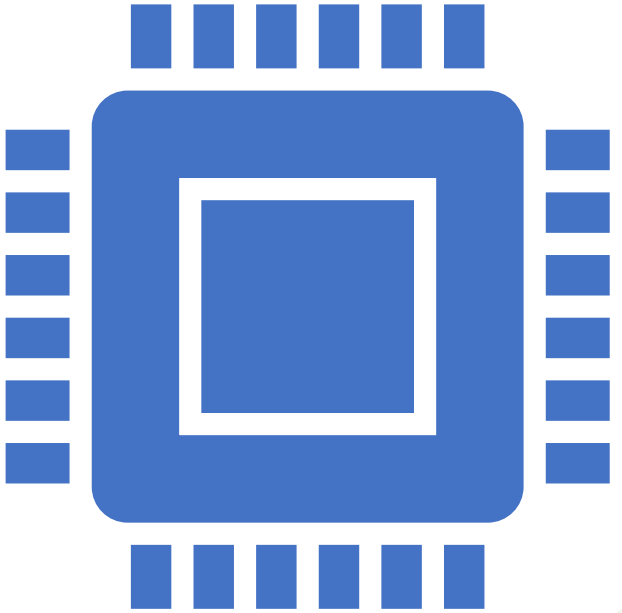


How does processor talk to cache?

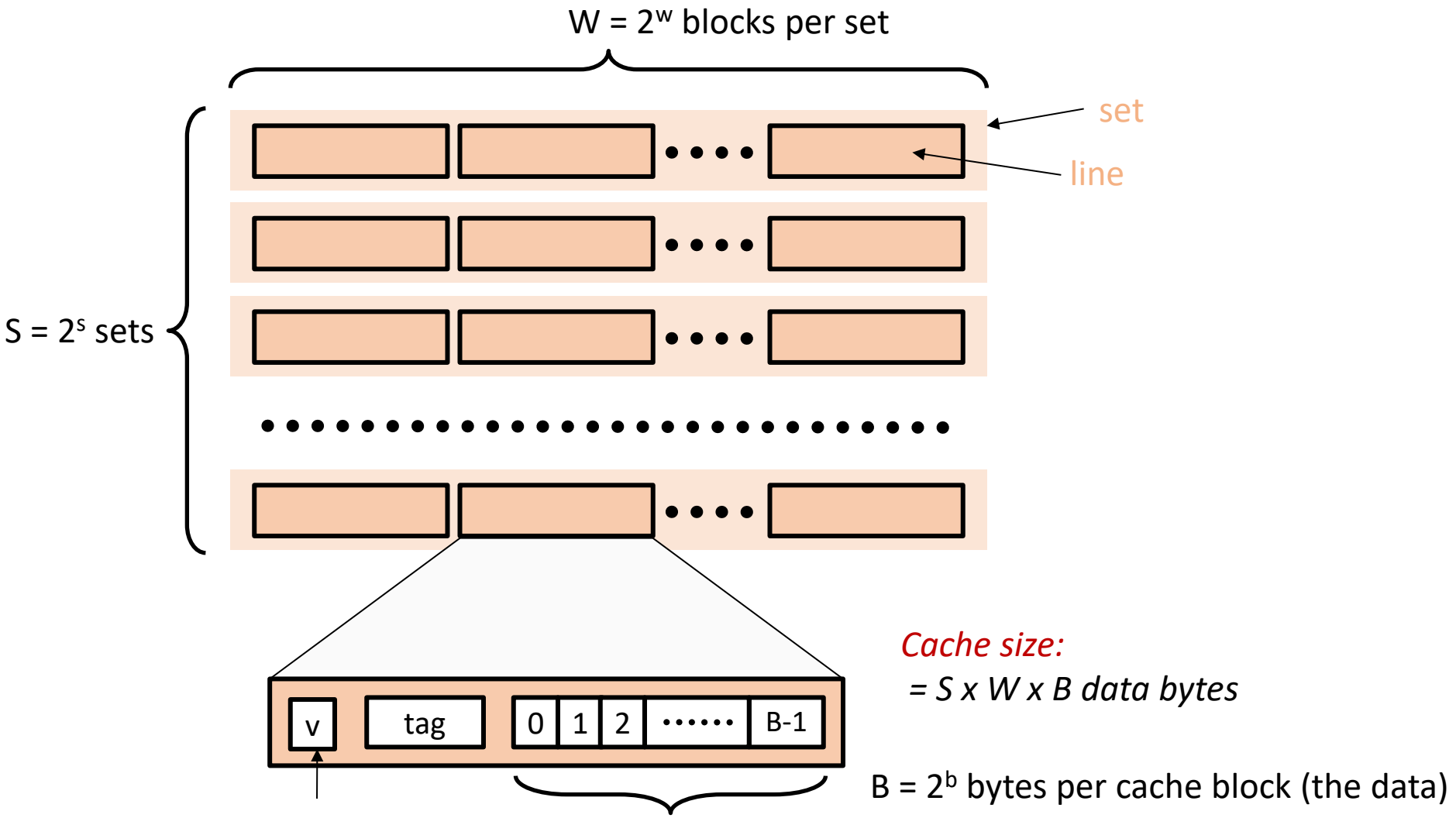
Same story

Processor provides an address

Cache provides the data

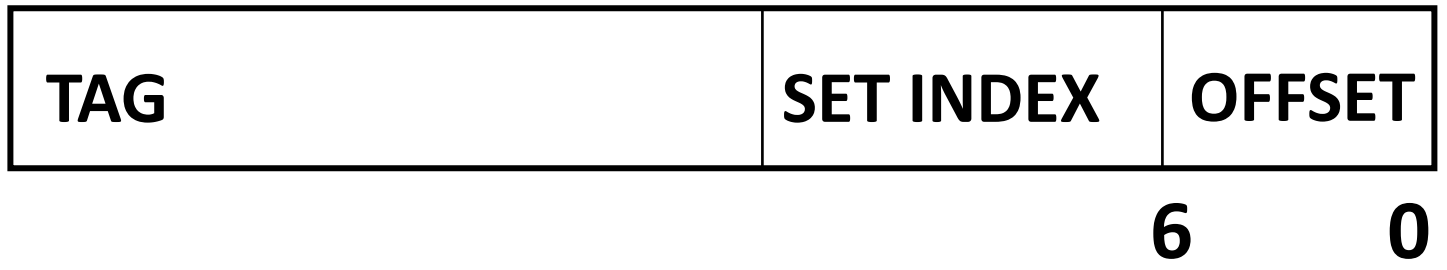


Set Associative



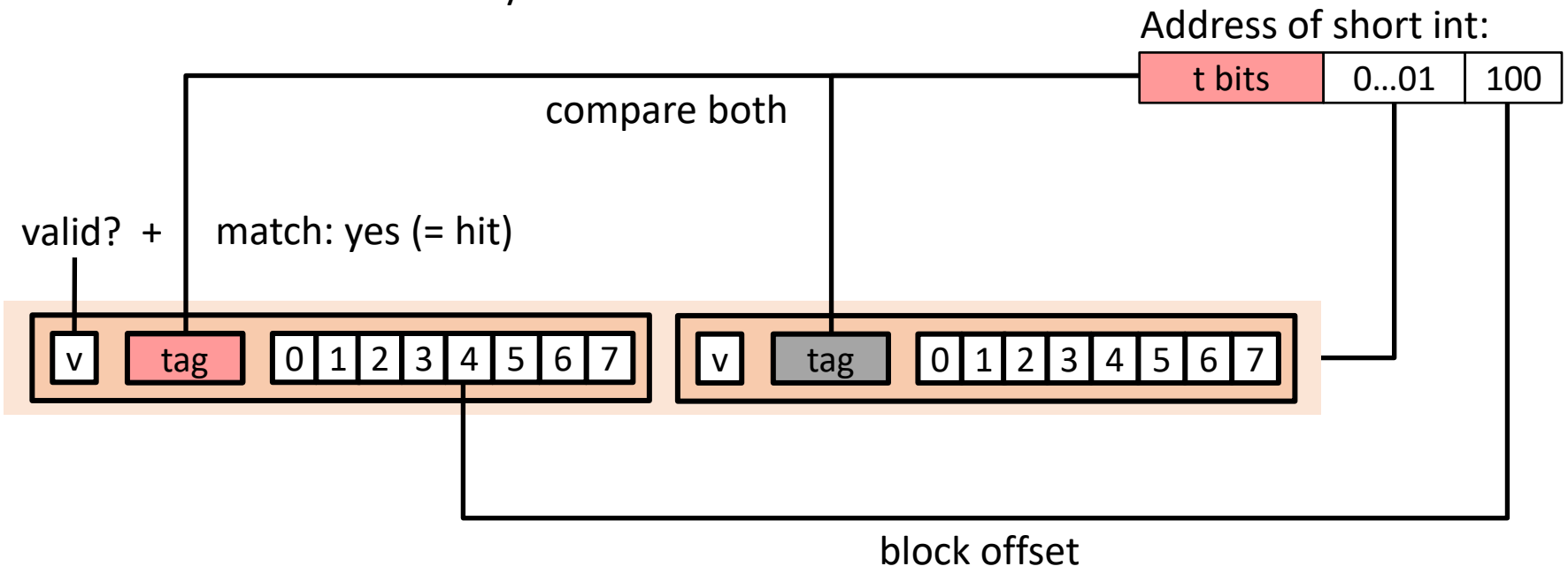
Cache size:
= S x W x B data bytes

Cache Mapping



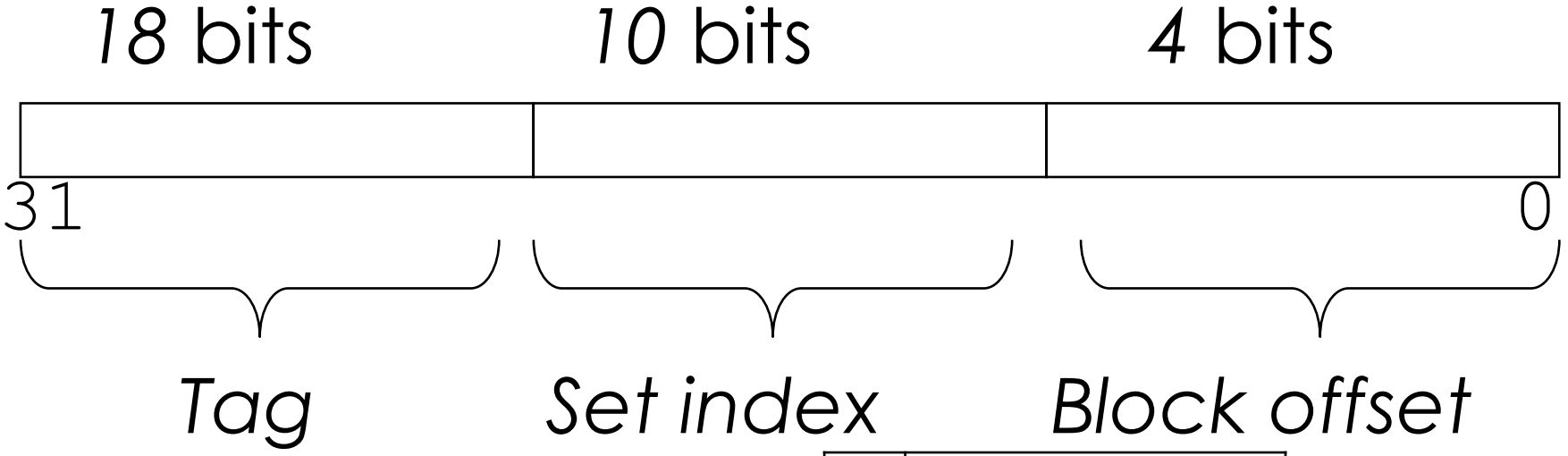
Set Associative in Action:

Way = 2: Two lines per set
Assume: cache block size B=8 bytes



Wake-up test again: #ints inside a block?

Address:



	# of int in block
A.	0
B.	1
C.	2
D.	4
E.	Unknown: We need more info

Wake-up test again:

If N = 16, how many bytes does the loop access of a?

```
int CS683(int* a, int N)
{
    int i;
    int sum = 0;
    for(i = 0; i < N; i++)
    {
        sum += a[i];
    }
    return sum;
}
```

	Accessed Bytes
A	4
B	16
C	64
D	256

The 3Cs

- Cold (compulsory) miss
  - Cold misses occur because the cache starts empty and this is the first reference

- Capacity miss
  - Occurs when the set of active cache blocks (working set) is larger than the cache.

- Conflict miss ☺ conflicting addresses into one set

Let’s try to make code cache-conscious

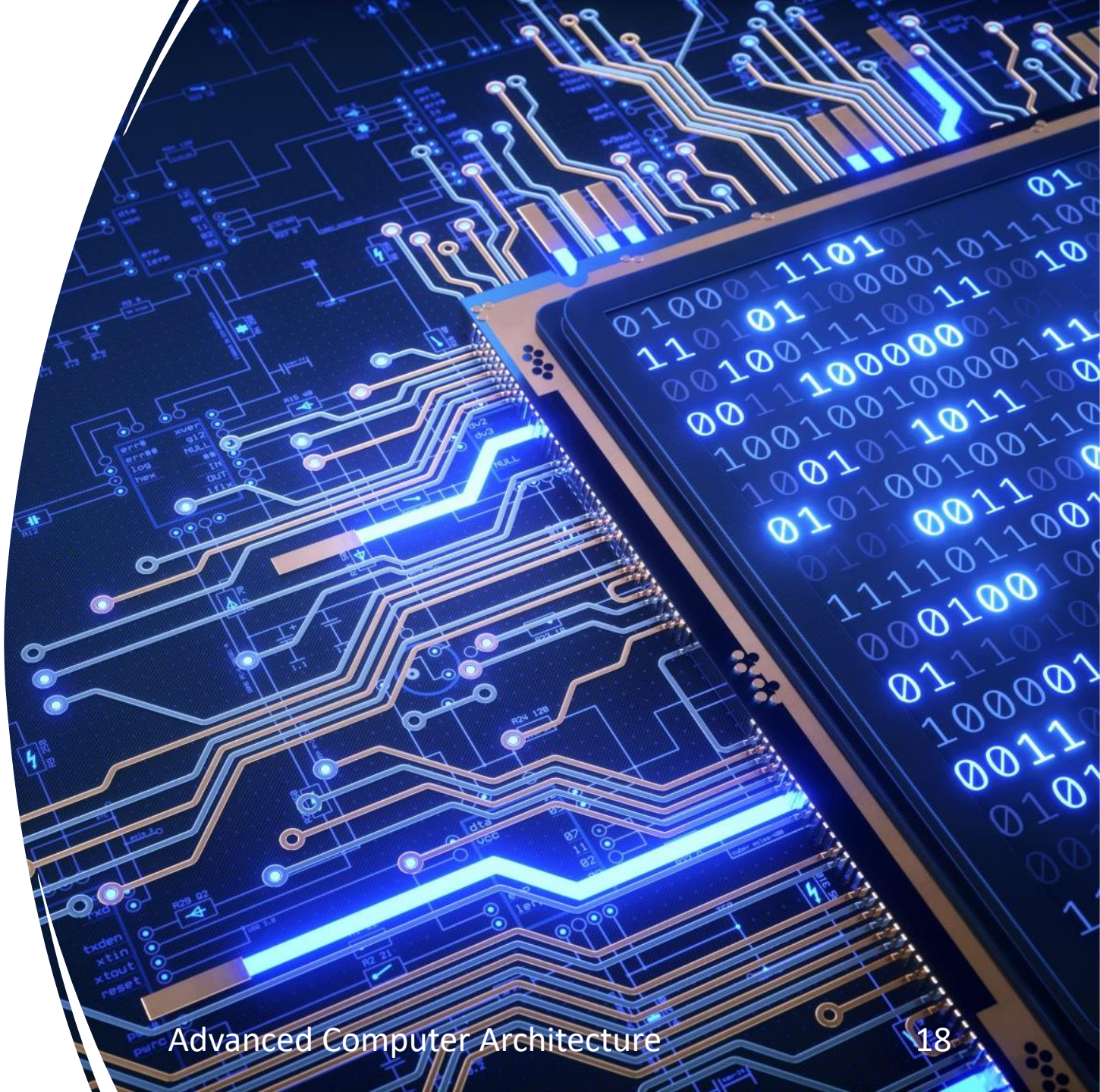
Matrix Multiplication

- Description:
  - Multiply *N* x *N* matrices
  - Matrix elements are doubles (8 bytes)
  - O(N^3) total operations

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

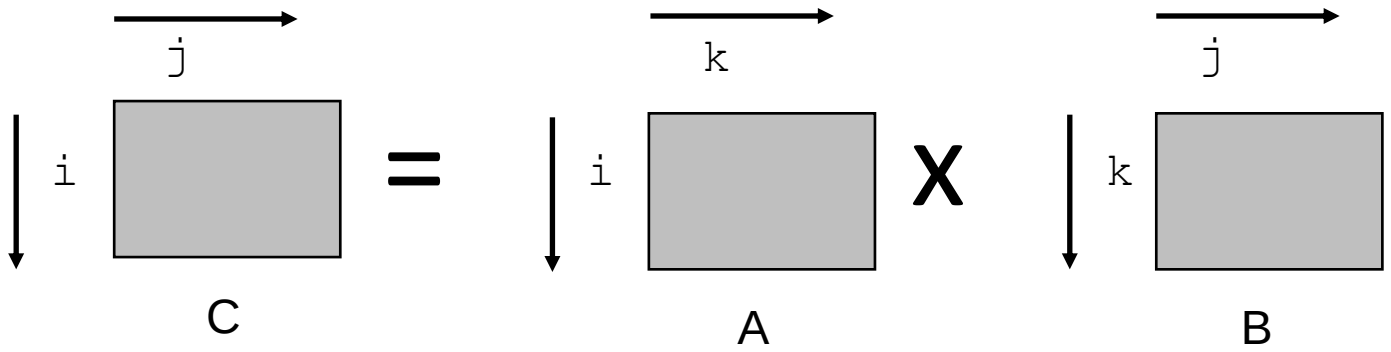
Why Matrix Multiplication?

- AI/ML
- Image Processing
- Scientific Computing
- Graph traversals
- Many more …



Miss Rate Analysis

- Assume:
  - Block size = 32B (big enough for four doubles)
  - Matrix dimension (N) is very large
    - Approximate 1/N as 0.0
  - Cache is not even big enough to hold multiple rows
- Analysis Method:
  - Look at access pattern of inner loop

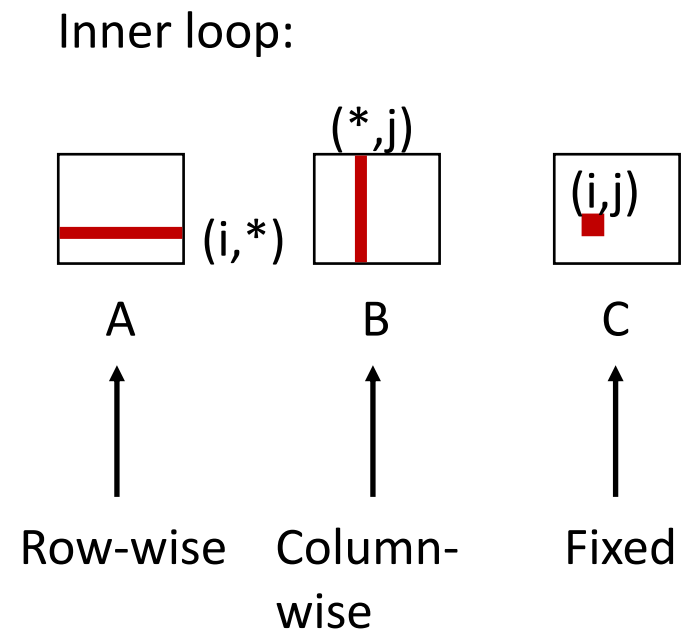


Cache Layout

- C arrays allocated in row-major order
  - each row in contiguous memory locations
- Stepping through columns in one row:
  - `for (i = 0; i < N; i++)`
    `sum += a[0][i];`
  - accesses successive elements
  - if block size (B) > sizeof(a_{ij}) bytes, exploit spatial locality
    - miss rate = sizeof(a_{ij}) / B
- Stepping through rows in one column:
  - `for (i = 0; i < N; i++)`
    `sum += a[i][0];`
  - accesses distant elements
  - no spatial locality!
    - miss rate = 1 (i.e. 100%)

Effect of loops (ijk)

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```



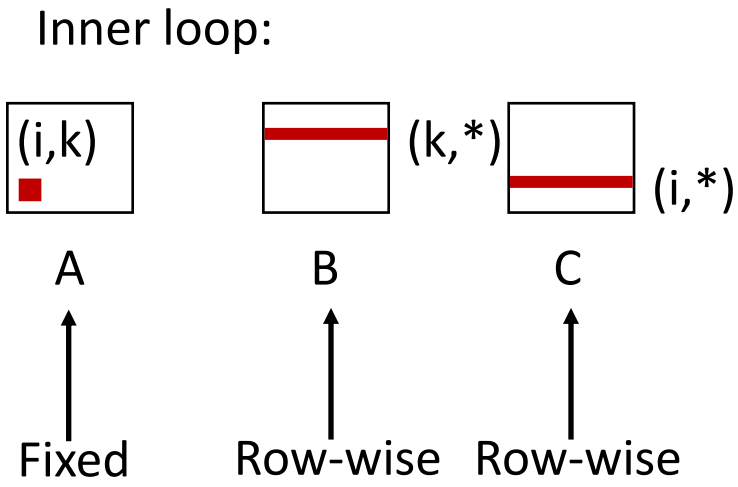
Block size = 32B (four doubles), your laptop will have 64B blocks

Miss rate for inner loop iterations:

A	B	C
0.25	1.0	0.0

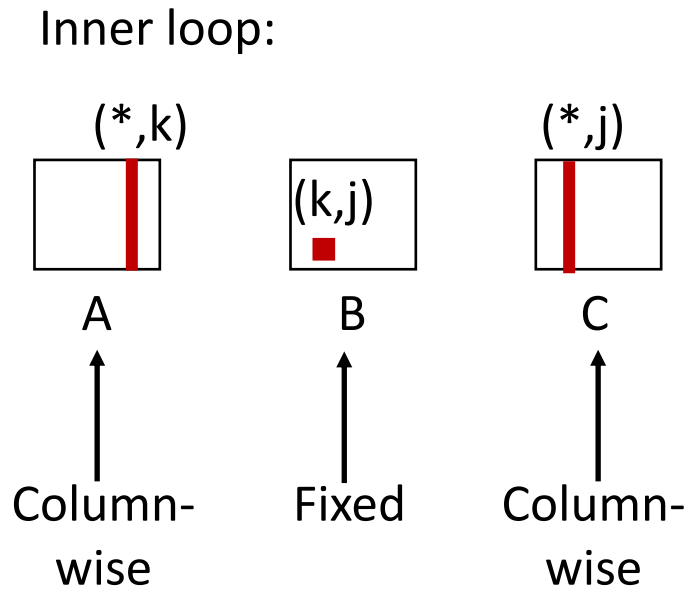
Effect of loops (kij)

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```



Effect of loops (jki)

```
/* jki */
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```



Summary

```
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
   sum = 0.0;
   for (k=0; k<n; k++)
     sum += A[i][k] * B[k][j];
   C[i][j] = sum;
 }
}
```

```
for (k=0; k<n; k++) {
 for (i=0; i<n; i++) {
  r = A[i][k];
  for (j=0; j<n; j++)
   C[i][j] += r * B[k][j];
 }
}
```

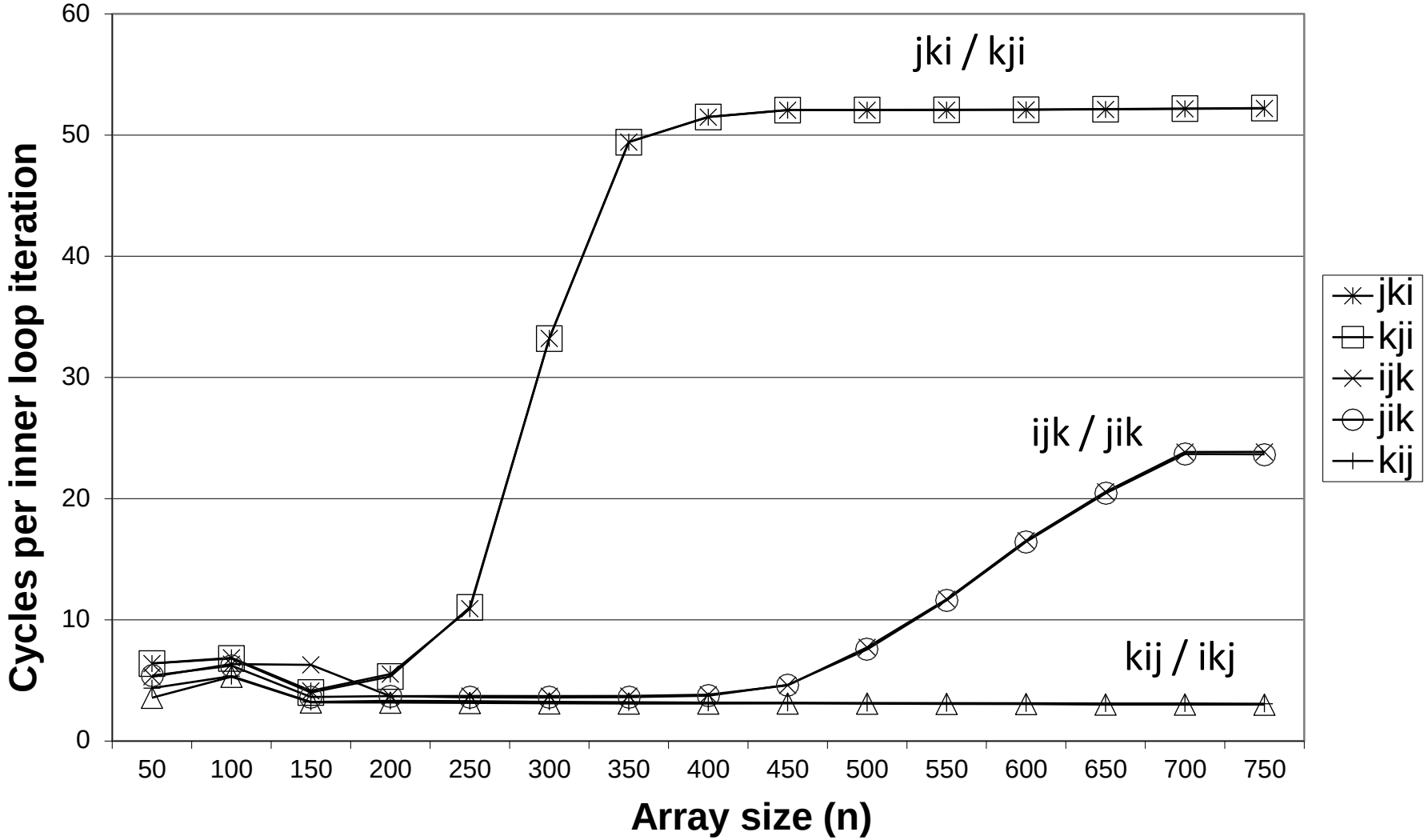
```
for (j=0; j<n; j++) {
 for (k=0; k<n; k++) {
   r = B[k][j];
   for (i=0; i<n; i++)
    C[i][j] += A[i][k] * r;
 }
}
```

ijk (& jik):
- 2 loads, 0 stores
- misses/iter = 1.25

kij (& ikj):
- 2 loads, 1 store
- misses/iter = 0.5

jki (& kji):
- 2 loads, 1 store
- misses/iter = 2.0

On an Intel Core i7



Devil is in the details?

1

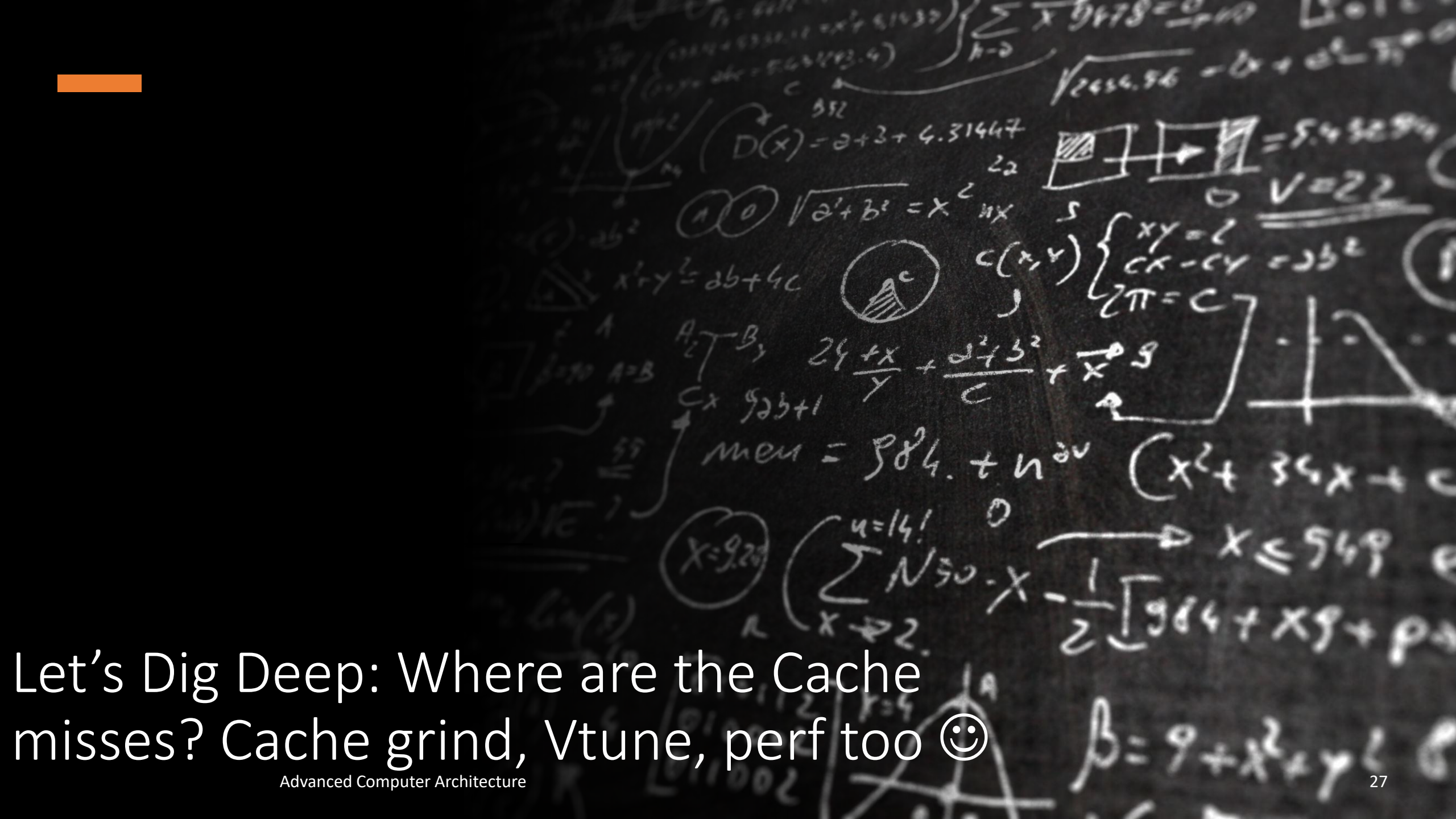
Metrics of interest: Cache performance

2

Hit time, hit rate, Miss rate, Miss latency

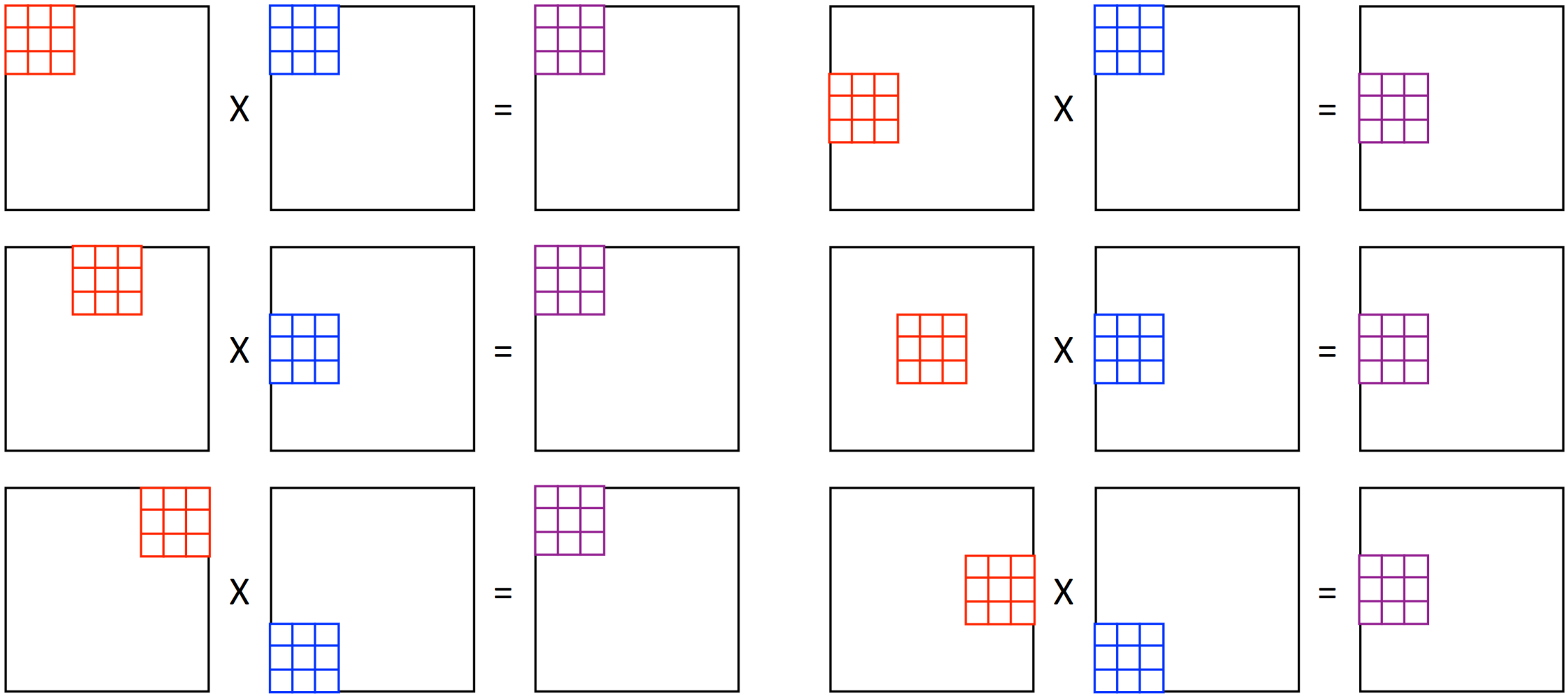
3

Average memory access time



Let’s Dig Deep: Where are the Cache misses? Cache grind, Vtune, perf too ☺

Matrix Multiplication with Tiling/Blocking: 101 ☺



A bit of analysis

- Number of slow memory references on unblocked matrix multiply
- Say a block size is 64 bytes and array elements are of 8 bytes

Here it is

9/8 (n^3) memory accesses

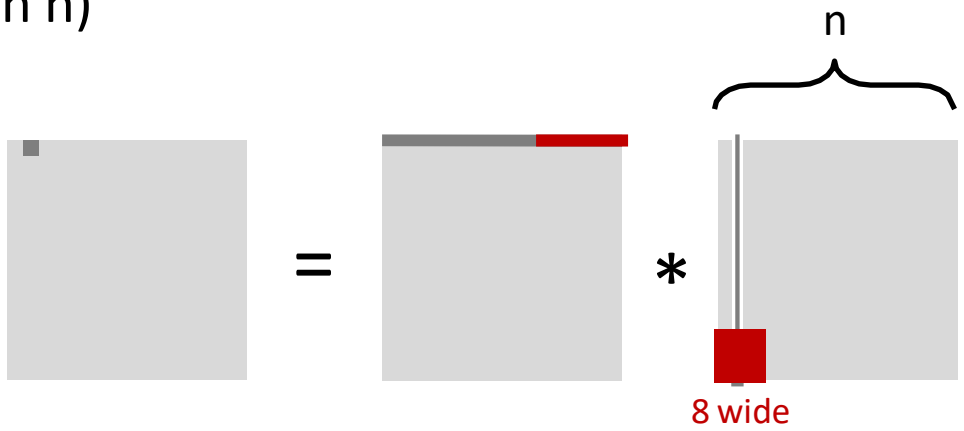
How: One miss in eight accesses (64B cache block) for the first array (1/8)

Second array: 8/8 misses, total: 9/8 mises per iteration

Cache Miss Analysis

- Assume:
  - Matrix elements are doubles
  - Cache block = 8 doubles
  - Cache size << n (much smaller than n)

- Second iteration:
  - $\frac{n}{8} + n = \frac{9n}{8}$ misses



- Total misses:
  - $\frac{9n}{8} * n^2 = \frac{9}{8}n^3$

The Code: Blocking one [Food for thought, read text book too]

```
/* Multiply n x n matrices a and b  */
void mmm(int n, double a[n][n], double b[n][n], double c[n][n]) {
    int i, j, k;
    for (i = 0; i < n; i+=B)
        for (j = 0; j < n; j+=B)
            for (k = 0; k < n; k+=B)
                /* B x B mini matrix multiplications */
                for (i1 = i; i1 < i+B; i1++)
                    for (j1 = j; j1 < j+B; j1++)
                        for (k1 = k; k1 < k+B; k1++)
                            c[i1][j1] += a[i1][k1]*b[k1][j1];
}
```

Blocked or Tiled MM

Number of blocks per row or per column= n/B,
For two arrays = 2n/B

Block size = B X B

Three blocks are in the C, $3B^2$ are in the cache

For each block: $B^2/8$ misses,

Two arrays = $2n/B \times B^2/8$

Total number of misses = $n^3/4B$

Confused?

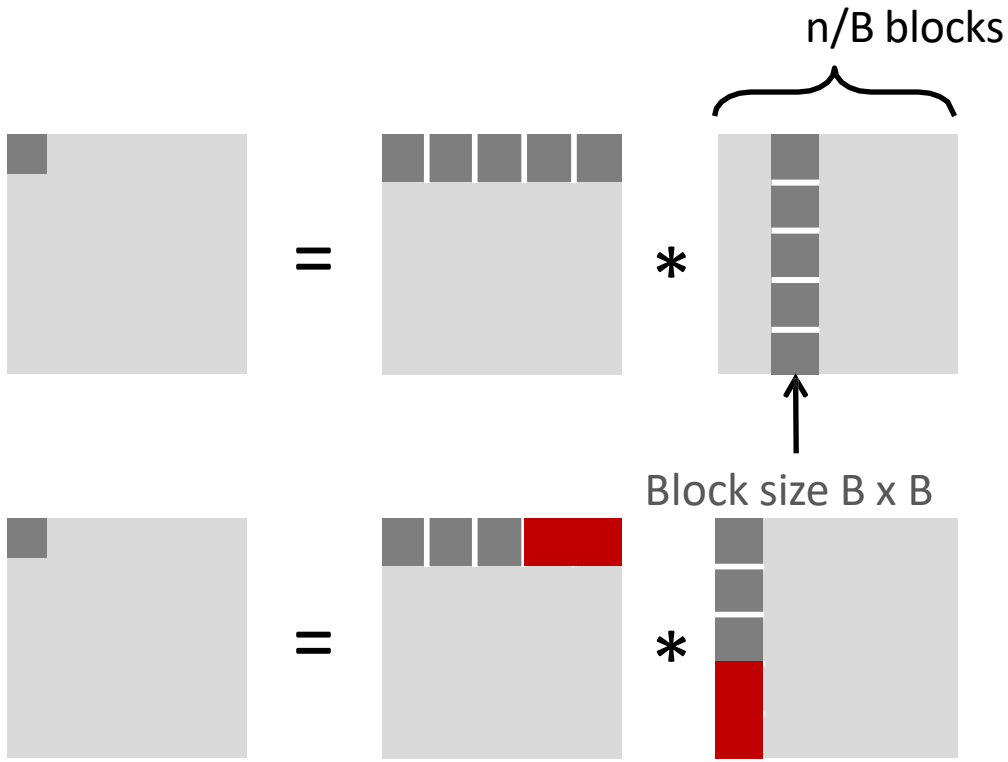
Cache Miss Analysis

- Assume:
  - Cache block = 8 doubles
  - Cache size << n (much smaller than n)
  - Three blocks ■ fit into cache, i.e., $3B^2 < C$

- First (block) iteration:
  - $\frac{B^2}{8}$ misses for each block (tile)
  - $2 * \frac{n}{B} * \frac{B^2}{8} = \frac{2*nB}{8}$

  (ignoring matrix C)

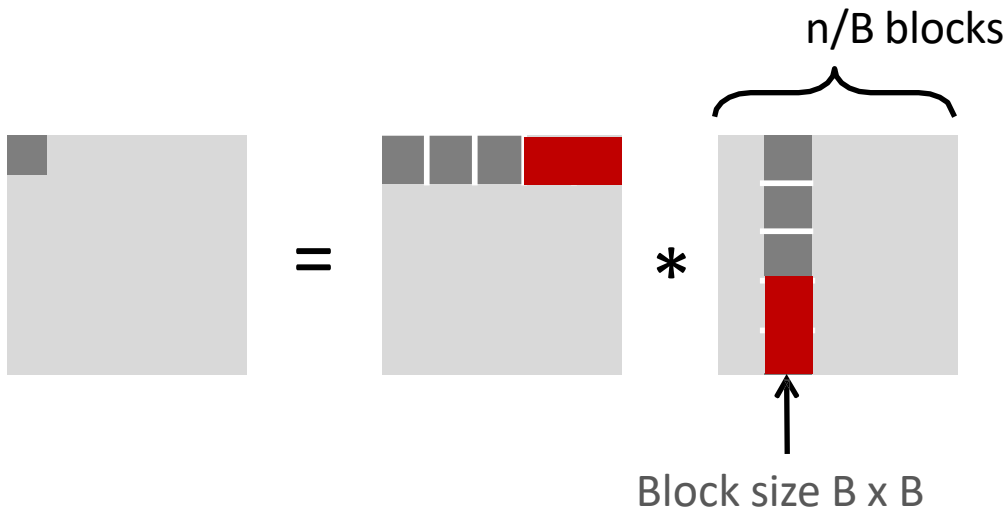
  - Afterwards in cache (schematic)



Cache Miss Analysis

- Assume:
  - Cache block = 8 doubles
  - Cache size << n (much smaller than n)
  - Three blocks ■ fit into cache, i.e., $3B^2 < C$

- Second (block) iteration:
  - Same as first iteration
  - $2 * \frac{n}{B} * \frac{B^2}{8} = \frac{2*nB}{8}$



- Total misses:
  - $\frac{2*nB}{8} * (\frac{n}{B})^2 = \frac{2*n^3}{8*B}$

Summary

Without blocking	With blocking
$\frac{9}{8} * n^3$	$\frac{2}{8B} * n^3$

- Find largest possible block size BLK, but limit $3BLK^2 < C$!
- Reason for dramatic difference is that matrix multiplication has inherent temporal locality
  - Input data is $3n^2$, computation is $2n^3$, and every array element is used $O(n)$ times!
  - But the program has to be written properly (choose good variant and BLK)